

Sistemas Lógicos Expertos en Free Pascal

Documento Técnico Comparativo

MOTORPASCAL vs. CSP-PARSER

Rodolfo Rodrigo

Departamento de Electromecánica, Facultad de Ingeniería

Universidad Nacional de San Juan (UNSJ)

Instituto de Automática (INAUT), UNSJ-CONICET

San Juan, Argentina

Marzo 2026

Resumen

Este documento presenta un análisis comparativo de dos sistemas lógicos implementados en Free Pascal: MOTORPASCAL, un evaluador completo de ecuaciones lógicas con generación de tablas de verdad, análisis de consistencia y extracción de features para redes neuronales; y CSP-PARSER, un pipeline modular de verificación y propagación de restricciones para problemas de *Constraint Satisfaction* (CSP) basado en el algoritmo AC-3 con aritmética de intervalos. Ambos sistemas comparten lenguaje de implementación, formato de entrada JSON y soporte para tipos integer, real, boolean y set, pero difieren fundamentalmente en su modelo computacional: MOTORPASCAL opera sobre *asignaciones puntuales* (evaluación directa), mientras que CSP-PARSER opera sobre *dominios* (propagación por restricciones). Se describen las arquitecturas, ejecutables, algoritmos centrales y posibles líneas de integración.

Índice

1. Introducción	2
2. MotorPascal: Evaluador de Ecuaciones Lógicas	2
2.1. Propósito	2
2.2. Arquitectura en Capas	2
2.2.1. Capa 1: Core	2
2.2.2. Capa 2: Engines	3
2.2.3. Capa 3: Main (Orquestadores)	3
2.2.4. Capa 4: Programs	3
2.3. Formato de Entrada	3
2.4. Operadores y Precedencia	4
2.5. Características Distintivas	4
3. CSP Parser: Pipeline de Propagación de Restricciones	4
3.1. Propósito	4
3.2. Arquitectura Pipeline (Filosofía Unix)	4
3.3. Etapas del Pipeline	5
3.3.1. Etapa 1: SyntaxChecker	5
3.3.2. Etapa 2: JsonToGraph	5
3.3.3. Etapa 3: FunctionChecker	5
3.3.4. Etapa 4: Motores de Propagación	5
3.4. Componentes de Bajo Nivel (Bibliotecas C)	5

3.5. Pratt Parser	6
3.6. Formato de Entrada	6
3.7. Características Distintivas	6
4. Análisis Comparativo	6
4.1. Tabla Comparativa General	6
4.2. Diferencia Fundamental: Evaluación vs. Propagación	6
4.3. Componentes Compartidos vs. Exclusivos	7
5. Casos de Uso	7
5.1. MotorPascal	7
5.2. CSP Parser	8
6. Posibles Líneas de Integración	8
7. Resumen Ejecutivo	9
A. Ejecutables de CSP Parser	9
B. Resumen de Archivos Fuente	9

1. Introducción

Los sistemas expertos basados en reglas lógicas constituyen una herramienta fundamental en áreas como diagnóstico médico, control de calidad industrial, planificación y verificación formal. Este documento describe dos proyectos complementarios desarrollados en Free Pascal que abordan el problema de la inferencia lógica desde perspectivas distintas:

1. **MotorPascal** — Evaluación directa de expresiones lógicas y aritméticas sobre variables con valores concretos, con generación exhaustiva de tablas de verdad y extracción de features para ANN.
2. **CSP-Parser** — Pipeline composable de propagación de restricciones sobre variables con *dominios*, utilizando consistencia de arco (AC-3), aritmética de intervalos y grid bitmask.

Ambos proyectos comparten: lenguaje (Free Pascal ≥ 3.2), formato de entrada (JSON), tipos de datos (integer, real, boolean, set), sistema de compilación (**Makefile**), y 9 ejecutables especializados cada uno.

2. MotorPascal: Evaluador de Ecuaciones Lógicas

2.1. Propósito

MOTORPASCAL recibe un archivo JSON con variables tipadas y un conjunto de ecuaciones, y produce resultados concretos: valores de verdad, resultados numéricos, tablas de verdad exhaustivas, análisis de consistencia (tautologías/contradicciones) y representaciones grafales para entrada a redes neuronales artificiales.

2.2. Arquitectura en Capas

El sistema se organiza en cuatro capas con dependencia estrictamente descendente:

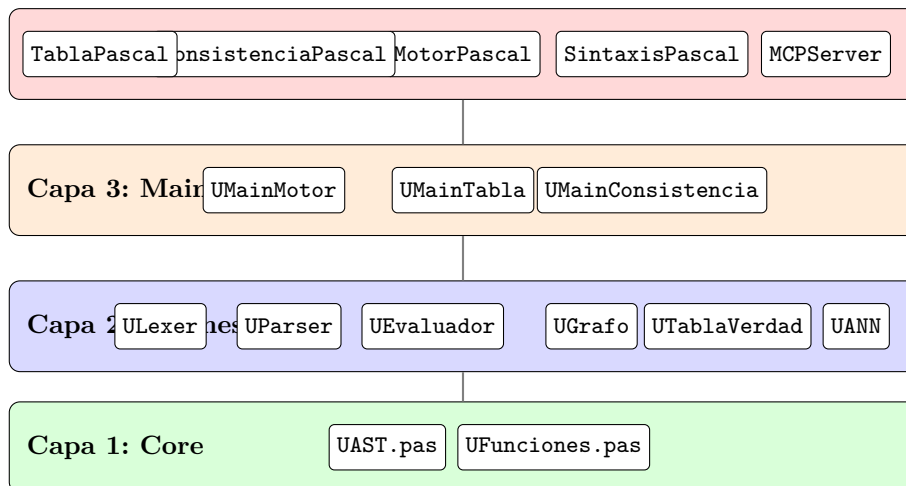


Figura 1: Arquitectura en capas de MOTORPASCAL.

2.2.1. Capa 1: Core

- **UAST.pas** (565 lín.) — Define **TNodo**, **TTipoNodo** (Numero, Identificador, Operador), **TTipoDato** (tdInteger, tdReal, tdBoolean, tdSet), y la estructura recursiva del AST.
- **UFunciones.pas** (180 lín.) — Utilidades matemáticas y de conversión de tipos.

2.2.2. Capa 2: Engines

- UVariables.pas (228 l n.) — Tabla de hasta 100 variables tipadas.
- UJSON.pas (505 l n.) — Parser JSON con soporte AnsiString ({\$H+}).
- ULexer.pas (410 l n.) — Tokenizaci n de ecuaciones.
- UParser.pas (1173 l n.) — Parser recursivo descendente con precedencia por niveles.
- UTypeChecker.pas (229 l n.) — Validaci n sem ntica de tipos.
- UEval ador.pas (538 l n.) — Recorrido del AST y c lculo de resultados.
- UGrafo.pas (353 l n.) — Matriz de adyacencia y features por nodo para ANN.
- UTablaVerdad.pas — Generaci n de tablas de verdad exhaustivas.
- UBacktracking.pas (466 l n.) — B squeda exhaustiva de asignaciones satisfactorias.
- UANN.pas, UANN_Engine.pas — Extracci n de features y motor de redes neuronales.

2.2.3. Capa 3: Main (Orquestadores)

Ocho unidades UMain*.pas que combinan engines para cada modo de operaci n.

2.2.4. Capa 4: Programs

Nueve ejecutables compilados (ver Tabla 1).

Cuadro 1: Ejecutables de MOTORPASCAL.

Ejecutable	Funci�n
MotorPascal	Eval�ador completo con grafo y ANN
SintaxisPascal	Validador sint�ctico r�pido
ConsistenciaPascal	Detector de tautolog�as/contradicciones
TablaPascal	Tablas de verdad (solo boolean)
BacktrackingPascal	B�squeda de soluciones (solo boolean)
TablaGen	Tablas de verdad (generalizado, 4 tipos)
BacktrackingGen	B�squeda de soluciones (generalizado)
ForwardPascal	Forward tracking evaluador
MCPServer	Servidor Model Context Protocol

2.3. Formato de Entrada

Listing 1: Ejemplo de entrada JSON para MOTORPASCAL.

```
1 {
2   "x": {"type": "integer", "value": 20},
3   "y": {"type": "integer", "value": 15},
4   "activo": {"type": "boolean", "value": true},
5   "sintomas": {
6     "type": "set",
7     "values": ["fiebre", "tos", "dolor_cabeza"]
8   },
9   "ecuaciones": [
10    "x > y",
11    "activo AND x > 10",
12    "fiebre IN sintomas"
13  ]
14 }
```

2.4. Operadores y Precedencia

La precedencia de operadores es (de mayor a menor):

Nivel	Operadores
9	NOT
8	*, /, div, mod, IN
7	+, -
6	>, <, >=, <=, =, <>
5	AND
4	OR

2.5. Características Distintivas

1. **Tablas de verdad exhaustivas:** Generación del producto cartesiano completo para todas las combinaciones de variables booleanas.
2. **Análisis de consistencia:** Detección automática de tautologías y contradicciones.
3. **Features para ANN:** Extracción de la matriz de adyacencia del AST y features por nodo (tipo, operador, profundidad) para alimentar redes neuronales.
4. **Servidor MCP:** Integración con modelos de lenguaje vía el Model Context Protocol.
5. **509 modelos de prueba** organizados en 82 categorías.

3. CSP Parser: Pipeline de Propagación de Restricciones

3.1. Propósito

CSP-PARSER implementa un pipeline modular para problemas de *Constraint Satisfaction* (CSP). A diferencia de MOTORPASCAL, las variables no tienen valores puntuales sino *dominios*: intervalos $[lo, hi]$ para numéricos, listas explícitas para enteros, y subconjuntos para booleanos y sets. El objetivo es *propagar restricciones* para reducir los dominios a un punto fijo consistente.

3.2. Arquitectura Pipeline (Filosofía Unix)

El diseño sigue la filosofía Unix: programas pequeños, cada uno hace una cosa bien, componibles vía JSON.

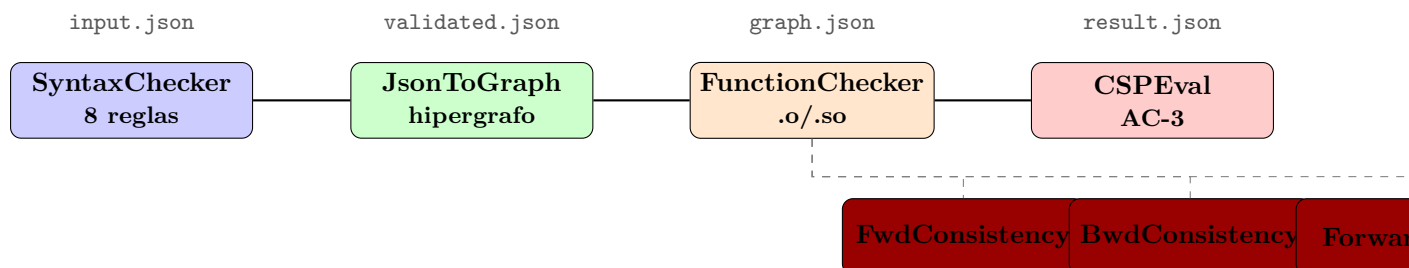


Figura 2: Pipeline de CSP-PARSER con motores de propagación alternativos.

3.3. Etapas del Pipeline

3.3.1. Etapa 1: SyntaxChecker

Valida la especificación JSON contra 8 reglas semánticas formales:

1. Variables declaradas sin usar → error.
2. Funciones declaradas sin usar → error.
3. Variables usadas sin declarar → error.
4. Funciones usadas sin declarar → error (excepto builtins).
5. Redefinición de variable → error.
6. Redefinición de función → error.
7. Tipo incorrecto en expresión → error.
8. Override de builtin → error.

3.3.2. Etapa 2: JsonToGraph

Convierte el sistema JSON validado en un hipergrafo de restricciones:

Listing 2: Salida de JsonToGraph (fragmento).

```
1 {  
2   "nodes": [{ "id": 0, "kind": "CT_ADD", "left": 1, "right": 2 }],  
3   "vars":  [{ "id": 0, "name": "x", "kind": "vkNumeric",  
4             "dom_lo": 0.0, "dom_hi": 100.0 }],  
5   "constrs": [{ "id": 0, "tree_id": 5, "var_refs": [0,1] }],  
6   "funcs":  [{ "id": 0, "name": "mifunc",  
7             "inputs": [...], "outputs": [...]}]  
8 }
```

3.3.3. Etapa 3: FunctionChecker

Verifica que toda función user-defined tenga su implementación compilada (.o o .so) en el search path.

3.3.4. Etapa 4: Motores de Propagación

Cuadro 2: Motores de propagación de CSP-PARSER.

Motor	Algoritmo	Descripción
CSPEval	AC-3 iterativo	Grid bitmask 20 puntos, punto fijo
FwdConsistency	AC-3 forward	Verificación hacia adelante
BwdConsistency	Proyección inversa	Por árbol de expresiones
ForwardChain	Encadenamiento	Trazado JSON paso a paso

3.4. Componentes de Bajo Nivel (Bibliotecas C)

CSP-PARSER delega las operaciones numéricas intensivas a bibliotecas C compiladas y enlazadas estáticamente:

- `minimath_interval.c` — Aritmética de intervalos: `IvMake`, `IvAdd`, `IvMul`, `IvDiv`, `IvContains`.
- `minimath_grid.c` — Filtros bitmask sobre grid de 20 puntos: `mg_narrow_gt`, `mg_narrow_ge`, etc.
- `minimath_trig.c`, `minimath_exp.c`, `minimath_util.c` — Funciones matemáticas.
- `minimath_complex.c` — Aritmética compleja.
- `roaring.c` — Roaring bitmaps para conjuntos comprimidos.

3.5. Pratt Parser

A diferencia del parser recursivo descendente de MOTORPASCAL, CSP-PARSER utiliza un *Pratt parser* (top-down operator precedence) implementado en `PrattParser.pas`. Este enfoque permite manejar la precedencia de operadores de forma más elegante y extensible, asociando funciones *nud* (null denotation) y *led* (left denotation) a cada token.

3.6. Formato de Entrada

Listing 3: Ejemplo de entrada JSON para CSP-PARSER.

```
1 {
2   "variables": [
3     {"name": "temperatura", "type": "numeric",
4      "domain": [0.0, 100.0], "value": [20.0, 25.0]},
5     {"name": "presion", "type": "set",
6      "domain": ["ALTA", "MEDIA", "BAJA"], "value": ["ALTA", "MEDIA"]},
7     {"name": "activo", "type": "boolean",
8      "domain": [true, false], "value": [true]}
9   ],
10  "expressions": [
11    {"constraints": [
12      "temperatura > 20.0",
13      "presion IN {ALTA, MEDIA}",
14      "activo AND (temperatura < 80.0)"
15    ]}
16  ]
17 }
```

3.7. Características Distintivas

1. **Propagación AC-3:** Reducción iterativa de dominios a punto fijo mediante consistencia de arco.
2. **Aritmética de intervalos:** Operaciones $[a, b] \oplus [c, d]$ con garantías de inclusión.
3. **Grid bitmask:** Discretización de dominios numéricos en 20 puntos con filtrado por máscara de bits.
4. **Funciones externas:** Soporte para funciones user-defined compiladas como objetos `.o/.so`.
5. **Filosofía Unix:** Cada etapa es un programa independiente, componible vía stdin/stdout en JSON.
6. **Forward chaining con trazado:** Registro paso a paso de la propagación para depuración y explicabilidad.

4. Análisis Comparativo

4.1. Tabla Comparativa General

4.2. Diferencia Fundamental: Evaluación vs. Propagación

La diferencia esencial entre ambos sistemas puede formalizarse como sigue. Sea $\mathcal{V} = \{v_1, \dots, v_n\}$ un conjunto de variables y $\mathcal{C} = \{c_1, \dots, c_m\}$ un conjunto de restricciones.

Cuadro 3: Comparación general entre MOTORPASCAL y CSP-PARSER.

Aspecto	MotorPascal	CSP-Parser
Problema central	Evaluación de expresiones con valores concretos	Propagación de restricciones sobre dominios
Modelo de datos	Variables $\mapsto$ valores puntuales	Variables $\mapsto$ dominios $[lo, hi]$, listas
Salida principal	Resultados, tablas de verdad, features ANN	Dominios reducidos, status de consistencia
Algoritmo central	Evaluación directa + enumeración exhaustiva	AC-3 + aritmética de intervalos
Parser	Recursivo descendente (UParser.pas)	Pratt parser (PrattParser.pas)
Arquitectura	En capas (core/engines/main/-programs)	Pipeline Unix composable
Nodos AST	~ 10 tipos	33 tipos (ExpressionAST.pas)
Bibliotecas C	No	Sí (interval, grid, trig, complex, roaring)
Funciones externas	No	Sí (verificación .o/.so)
Red neuronal	Sí (UANN, features de grafo)	No
Servidor MCP	Sí (MCPServer)	No
Modelos de prueba	509 (82 categorías)	Ejemplos en json/ y test/

MotorPascal (evaluación puntual). Dada una asignación completa $\sigma : \mathcal{V} \rightarrow \text{Val}$, MOTORPASCAL computa:

$$\forall c_j \in \mathcal{C} : \text{eval}(c_j, \sigma) \in \{\text{true}, \text{false}, v \in \mathbb{R}\} \quad (1)$$

En el caso booleano, puede enumerar exhaustivamente todas las $\sigma \in \text{Val}^n$ (tabla de verdad).

CSP Parser (propagación de dominios). Dada una asignación de dominios $D : \mathcal{V} \rightarrow 2^{\text{Val}}$, CSP-PARSER computa el operador de punto fijo:

$$D^* = \text{AC-3}(D, \mathcal{C}) \quad \text{donde} \quad D_i^* \subseteq D_i \quad \forall i, \quad D^* = \text{AC-3}(D^*, \mathcal{C}) \quad (2)$$

El resultado D^* es el mayor conjunto de dominios arc-consistente contenido en D .

4.3. Componentes Compartidos vs. Exclusivos

5. Casos de Uso

5.1. MotorPascal

- **Diagnóstico médico:** Evaluar un conjunto de síntomas contra reglas clínicas y determinar si se satisfacen criterios diagnósticos.
- **Verificación de soldadura:** Validar que los parámetros de un proceso de soldadura cumplen con las ecuaciones de calidad.
- **Generación de datos para ML:** Usar las tablas de verdad y features de grafo como datasets para entrenar clasificadores.

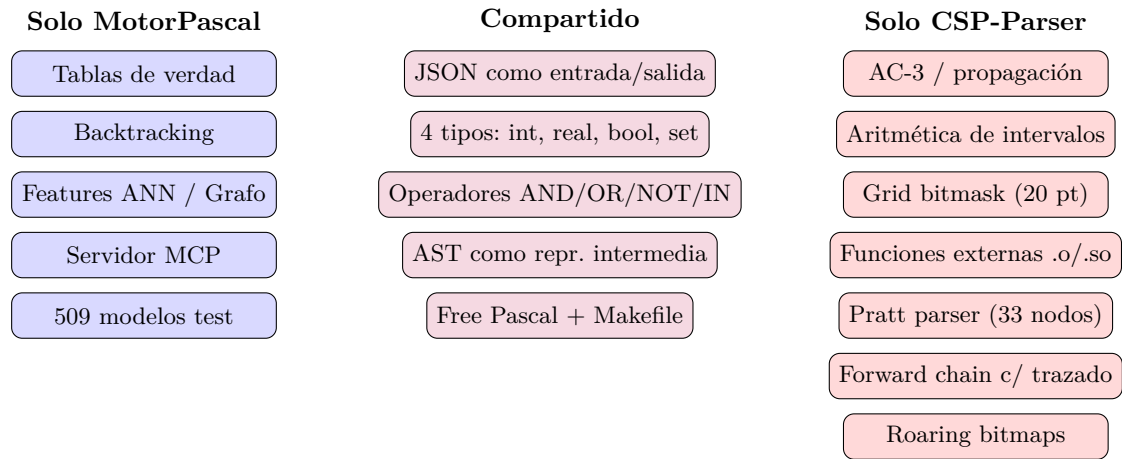


Figura 3: Diagrama de Venn funcional: componentes compartidos y exclusivos.

- **Integración con LLMs:** Vía el servidor MCP, un modelo de lenguaje puede invocar evaluaciones lógicas en tiempo real.

5.2. CSP Parser

- **Planificación industrial:** Dado un conjunto de variables de proceso con rangos admisibles y restricciones de operación, determinar los dominios factibles.
- **Configuración de productos:** Reducir las opciones válidas de un configurador combinatorio.
- **Verificación de modelos:** Comprobar consistencia de un sistema de restricciones antes de su implementación.
- **Explicabilidad:** El forward chaining con trazado permite explicar *por qué* un dominio se redujo.

6. Posibles Líneas de Integración

Dado que ambos sistemas comparten formato JSON y tipos de datos, se identifican las siguientes oportunidades de integración:

1. **Pipeline CSP → MotorPascal:** Usar CSP-PARSER para reducir dominios, luego alimentar MOTORPASCAL con las asignaciones puntuales resultantes para evaluación y generación de features ANN.
2. **MCP unificado:** Extender el MCPServer de MOTORPASCAL para invocar también los ejecutables del pipeline CSP, ofreciendo un punto de acceso único a ambos sistemas.
3. **Pratt parser compartido:** Migrar MOTORPASCAL al Pratt parser de CSP-PARSER para unificar el front-end de parsing con soporte para los 33 tipos de nodo.
4. **ANN sobre dominios:** Usar las features de grafo de MOTORPASCAL no solo sobre ASTs evaluados sino también sobre los hipergrafos CSP generados por CSP-PARSER.

Cuadro 4: Resumen comparativo en una línea.

	MotorPascal	CSP-Parser
Pregunta que responde	¿Qué es verdadero dado σ ?	¿Qué es factible dado D ?
Espacio de operación	Punto en Val^n	Volumen en $\prod D_i$
Paradigma	Evaluación	Inferencia
Complejidad	$O(2^n)$ para tablas de verdad	$O(e \cdot d^3)$ para AC-3

7. Resumen Ejecutivo

En síntesis, MOTORPASCAL y CSP-PARSER son dos caras del mismo problema: MOTORPASCAL *evalúa* (dado un punto en el espacio de estados, ¿qué es verdadero?), mientras que CSP-PARSER *inferi* (dado un volumen en el espacio de estados, ¿cómo se reduce por las restricciones?). Juntos, constituyen un framework completo para razonamiento lógico: desde la especificación y verificación de restricciones hasta la evaluación concreta y la generación de datos para aprendizaje automático.

A. Ejecutables de CSP Parser

Cuadro 5: Ejecutables de CSP-PARSER.

Ejecutable	Función
SyntaxChecker	Validación semántica (8 reglas)
JsonToGraph	Conversión JSON $\rightarrow$ hipergrafo CSP
FunctionChecker	Verificación de funciones externas
CSPEval	Propagador AC-3 con grid bitmask
FwdConsistency	Consistencia hacia adelante
BwdConsistency	Proyección inversa por AST
ForwardChain	Encadenamiento con trazado paso a paso
JsonSource	Entrada desde SQLite $\rightarrow$ JSON
JsonSink	Salida JSON $\rightarrow$ SQLite

B. Resumen de Archivos Fuente

Proyecto	Archivo	Rol
MOTORPASCAL	UAST.pas	Definición del AST y tipos
MOTORPASCAL	ULexer.pas	Análisis léxico
MOTORPASCAL	UParser.pas	Parser recursivo descendente
MOTORPASCAL	UEvaluador.pas	Evaluación de expresiones
MOTORPASCAL	UGrafo.pas	Matriz de adyacencia para ANN
MOTORPASCAL	UTablaVerdad.pas	Tablas de verdad exhaustivas
MOTORPASCAL	UBacktracking.pas	Búsqueda de soluciones
MOTORPASCAL	UANN.pas	Features para redes neuronales
CSP-PARSER	ExpressionAST.pas	33 tipos de nodo AST
CSP-PARSER	PrattParser.pas	Pratt parser (precedencia)

Proyecto	Archivo	Rol
CSP-PARSER	SyntaxChecker.pas	8 reglas semánticas
CSP-PARSER	JsonToGraph.pas	Construcción de hipergrafo
CSP-PARSER	CSPEval.pas	Propagador AC-3
CSP-PARSER	ForwardChain.pas	Forward chaining con trazado
CSP-PARSER	minimath_interval.c	Aritmética de intervalos
CSP-PARSER	minimath_grid.c	Grid bitmask (20 puntos)
CSP-PARSER	roaring.c	Roaring bitmaps